

15 Types of Databases and When to Use Them

#4 System Design - Types of Databases



ASHISH PRATAP SINGH

MAR 24, 2024

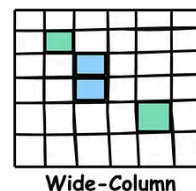
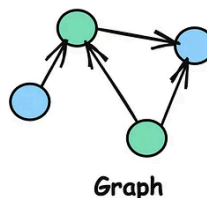
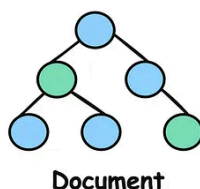
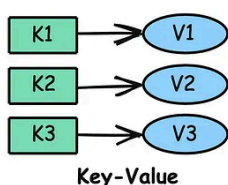
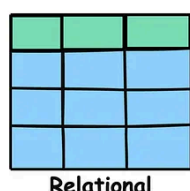


55

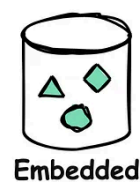
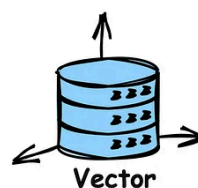
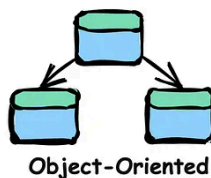
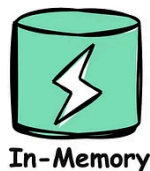


Share

Databases are foundational building blocks of System Design used to store, manage and retrieve data efficiently.



blog.algomaster.io



In this article, we'll explore 15 common types of databases and discuss when to use them, along with examples.

1. Relational Databases (RDBMS)

Relational databases structure data into one or more tables of rows and columns, with a unique key identifying each row. Rows in a table can be linked to rows in other tables through foreign keys, establishing a relationship between them.

This structure allows relational databases to handle large amounts of structured data, enforce data integrity, and support complex queries and ACID transactions.

They use Structured Query Language (SQL) for defining, manipulating, and querying data, making them highly versatile and widely used in various applications.

Common Use-cases:

- **Enterprise Applications:** For managing customer data, inventory, employee records, and financial transactions, where data integrity and relationships are critical.
- **E-commerce Platforms:** Handling product catalogs, customer orders, and payment transactions, requiring complex queries and transaction processing.
- **Banking and Financial Services:** Managing accounts, transactions, and user data, where the ACID properties ensure the reliability and consistency of financial operations.

Examples: MySQL, PostgreSQL, Oracle Database.

2. Key-Value Store

Key-value stores are NoSQL databases that store data as key-value pairs providing fast retrieval of values based on unique keys.

They are primarily used when the data model is based on key-value pairs and requires high scalability, availability and throughput.

However, they may not be the best fit for applications that require complex querying, data relationships, or strong consistency guarantees.

Common Use-cases:

- **Session Storage:** Storing and managing user session information such as user preferences, shopping carts or authentication tokens in web applications.
- **Caching:** Implementing caching mechanisms to improve the performance of web applications by storing frequently accessed data in memory for rapid retrieval.
- **Real-time data processing:** Key-value stores can quickly store and retrieve data for real-time analytics, event processing, or message queues.

Examples: Redis, DynamoDB.



[Become a Better Software Engineer - CodeCrafters.io](https://codecrafters.io)

Build your own Redis Learn about TCP servers, the Redis protocol and more 55 stages	Build your own HTTP server Learn about TCP servers, the HTTP protocol and more 11 stages
Build your own Interpreter Learn about tokenization, ASTs, tree-walk interpreters and more. FREE DURING BETA	Build your own Shell Learn about parsing shell commands, executing programs and more 12 stages
Build your own Git Learn about git objects, plumbing commands and more 7 stages	Build your own DNS server Learn about the DNS protocol, DNS record types and more. 8 stages

[CodeCrafters](https://codecrafters.io) is a YC backed platform that offers a unique, **hands-on** approach to practice **complex programming challenges** like building your own **Redis**, **Git**, **SQLite**, **Bittorrent client** and more from scratch.

3. Document Databases

Document databases, a subset of the broader NoSQL family, are designed to store, manage, and retrieve document-oriented information.

These databases handle data in a semi-structured format, typically JSON, XML, or BSON, allowing for a more flexible schema than traditional relational databases.

Document databases are particularly useful in scenarios where the data model evolves frequently, and where fast read and write performance is critical.

However, they may not be the best fit for highly structured data or complex transactions spanning multiple documents or collections.

Common Use-cases:

- **E-commerce Platforms:** Storing product catalogs with diverse attributes, user reviews, and inventory data, allowing for flexible representation of product information.
- **Content Management Systems (CMS):** Ideal for managing articles, user profiles, and comments, where each piece of content can be stored as a document.
- **Real-Time Analytics and IoT:** Handling varied data structures generated by IoT devices and supporting real-time analytics on this data.

Examples: MongoDB, Couchbase, and Apache CouchDB

4. Graph Databases

Graph databases are a type of NoSQL database that specialize in storing, managing, and querying complex networks of interconnected data.

They represent data as graphs, consisting of nodes (entities), edges (relationships between entities), and properties (information associated with nodes and edges).

By leveraging the graph structure, graph databases enable efficient traversal, querying, and analysis of interconnected data.

They are very useful in applications like social networks and recommendation engines.

Common Use-cases:

- **Social Networks:** Managing user profiles and their connections, enabling features like friend recommendations and social graph analysis.
- **Recommendation Systems:** Analyzing customer preferences, product inventories, and purchase histories to generate personalized product or content recommendations.
- **Knowledge Graphs:** Building vast repositories of interconnected data for semantic searches, information retrieval, and decision support systems.

Examples: Neo4j, Amazon Neptune.

5. Wide-Column Stores

Wide-Column Stores represent a type of NoSQL database optimized for storing and querying large amounts of data across many machines.

They organize data into tables with a flexible and dynamic column structure. They are designed to handle large-scale, distributed data storage and provide high scalability and performance.

Their column-oriented architecture, flexible schema, and eventual consistency model make them well-suited for applications that require high write throughput and real-time data processing.

However, they may not be the best fit for use cases that require complex joins, strong consistency, or strict ACID transactions.

Common Use-Cases:

- **Web analytics and user tracking:** Ideal for capturing and analyzing event data in real-time, such as web analytics, user activity logs, and network monitoring.
- **Real-Time Analytics:** They can quickly aggregate and analyze data, making them suitable for dashboards, alerting systems, and operational analytics.

Examples: Apache Cassandra, Apache HBase, Google Bigtable.

6. In-Memory Databases

In-Memory Databases store data directly in the main memory (RAM) of the computer, as opposed to disk-based storage.

They are designed to provide extremely fast data access and low latency by eliminating the need for disk I/O operations.

In-memory databases are particularly well-suited for applications that require real-time processing, high-speed transactions, and low-latency data access such as caching, real-time analytics, high-frequency trading.

However, they are costly and the main memory may lack sufficient capacity to store the entire dataset.

Common Use-Cases:

- **Online Gaming:** To manage user sessions and game state in real time, ensuring fast and responsive gameplay experiences.
- **High-Frequency Trading:** They enable a large number of financial transactions per second with minimal latency.

Examples: Redis, Memcached.

7. Time-Series Databases

Time-Series Databases (TSDBs) specialize in storing, retrieving, and managing time-stamped or time-series data.

Time-series data is a sequence of data points collected over time intervals.

TSDBs are commonly used in applications that generate and process time-series data, such as monitoring systems, sensor networks, financial trading platforms, and IoT (Internet of Things) devices.

They provide the necessary performance, scalability, and specialized features to handle the unique characteristics of time-series data.

Common Use-Cases:

- **Financial Trading Platforms:** For tracking stock prices, trade volumes, and market indicators over time, enabling trend analysis and algorithmic trading strategies.
- **IoT and Sensor Data Management:** Collecting and analyzing data from sensors and IoT devices, useful in smart homes, industrial automation, and environmental monitoring.
- **Performance Monitoring:** In IT and network infrastructure, to monitor system metrics (CPU usage, memory consumption, network traffic) over time, helping in capacity planning and anomaly detection.

Examples: InfluxDB, TimescaleDB, Prometheus.

Subscribe to receive new articles every week.

8. Object-Oriented Databases

Object-oriented databases (OODB) are databases that store and manipulate data as objects.

These objects are instances of classes, which can encapsulate both data (attributes) and behaviors (methods), mirroring the structure and concepts of object-oriented programming languages like Java, C++, or Python.

OODBs are particularly well-suited for applications where complex data models are necessary, or the application logic heavily relies on object-oriented principles.

By allowing developers to work directly with objects in the database, OODBs can simplify the development process and provide a more natural and efficient way to manage complex data structures and relationships.

Common Use-Cases:

- **Object-Oriented Applications:** Applications developed using OOP languages that require a seamless persistence mechanism for storing and retrieving objects without the need to convert them to a different format (object-relational mapping).
- **Multimedia Databases:** Storing, organizing, and retrieving multimedia items like images, videos, and audio files, which can benefit from the encapsulation of both data and behaviors (e.g., methods to play or edit).

Examples: ObjectDB, db4o

9. Text Search Databases

Text Search Databases are specialized systems designed for efficient storage, indexing, and retrieval of large volumes of unstructured or semi-structured text data.

They provide fast and scalable search capabilities, enabling users to query and find relevant information from vast collections of documents, web pages, or other text-based content.

Common Use-Cases:

- **E-commerce:** For product searches within online stores, helping customers find products based on descriptions, reviews, and metadata.
- **Web search:** These are used in Search engines like Google, Bing, and DuckDuckGo to index and search the vast amount of content available on the internet, allowing users to find relevant web pages based on their queries.
- **Log analysis:** These can be used to index and search large volumes of log data, such as application logs or system logs, for troubleshooting, monitoring, and analytics purposes.

Examples: Elasticsearch, Apache Solr, Sphinx.

10. Spatial Databases

Spatial databases are designed to store, manage, and analyze data that represents geographical or spatial information. They extend traditional database capabilities to handle complex spatial data types, such as points, lines, polygons, and other geometric shapes, along with their associated attributes and relationships.

Spatial databases employ efficient indexing techniques, such as R-trees or quadrees, to optimize spatial queries and improve performance.

They are used heavily in services that are based on the user's location, such as mapping routes, finding nearby restaurants, or tracking vehicle movements in real-time.

Common Use-Cases:

- **Geographic Information Systems (GIS):** For mapping, analyzing, and managing data related to places on the earth's surface for urban planning, environmental management, and emergency response planning.
- **Location-Based Services (LBS):** To provide services based on the user's location, such as mapping routes and finding nearby restaurants.
- **Logistics and transportation:** Spatial databases are used in logistics and transportation systems to optimize routes, track vehicle movements, and analyze traffic patterns.

Examples: PostGIS (extension for PostgreSQL), Oracle Spatial.

11. Blob Datastore

Blob (Binary Large Object) datastores are designed for storing, managing, and retrieving large blocks of unstructured data, such as images, audio files, videos, and documents.

Unlike traditional databases that handle structured data with well-defined fields and records, blob datastores are optimized for large, complex blobs of data that do not fit neatly into standard database schemas.

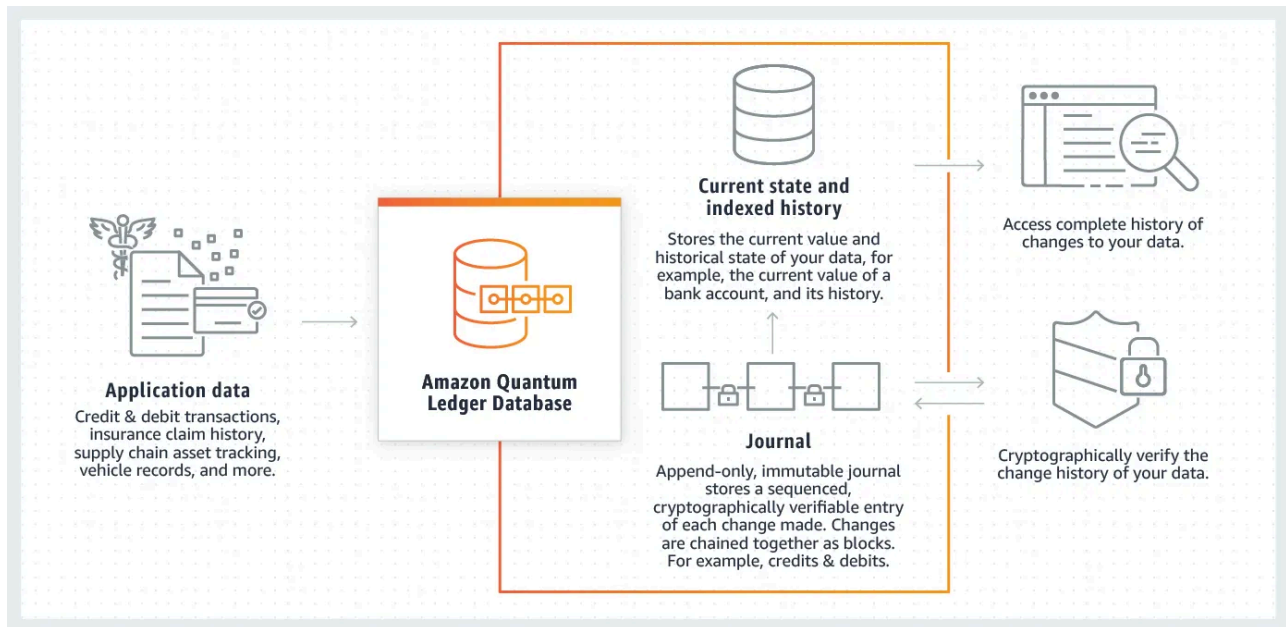
They provide a scalable, highly available, durable and cost-effective solution for managing massive amounts of unstructured data.

Common Use-Cases:

- **Content Delivery Networks (CDNs):** For storing and delivering large media files, such as videos and images, to users around the globe.
- **Big data storage:** Blob datastores can store large datasets, such as log files, sensor data, and scientific data, for big data analytics and processing pipelines.
- **Backup and archival:** Blob datastores provide a reliable and scalable solution for storing backup data, archives, and long-term data retention. They also offer cost-effective storage options for infrequently accessed data.

Examples: Amazon S3, Azure Blob Storage, HDFS.

12. Ledger Databases



Credit: <https://aws.amazon.com/qlldb/>

Ledger databases also known as blockchain databases are designed to provide an immutable, append-only record of transactions.

They are engineered to ensure that once a transaction is recorded, it cannot be altered or deleted, providing a verifiable and tamper-evident history of all changes over time.

Ledger databases are particularly suitable for applications that require a high level of trust, transparency, and immutability.

Common Use-Cases:

- **Supply chain management:** Ledger databases can track the movement of goods and materials across a supply chain, ensuring transparency and traceability.
- **Healthcare:** Managing patient records, consent forms, and treatment histories with a clear, unchangeable record.
- **Voting systems:** Ledger databases can provide a secure and transparent platform for conducting voting processes by ensuring the integrity of the voting results and prevent tampering or manipulation.

Examples: Amazon Quantum Ledger Database (QLDB), Hyperledger Fabric.

13. Hierarchical Databases

Hierarchical databases organize data into a tree-like structure where data is stored in records, and each record has a single parent record but can have a multiple children, establishing a one-to-many relationship between records.

Hierarchical databases were popular in the early days of computing, particularly in mainframe systems. They were commonly used for file systems, where directories and files naturally fit into a hierarchical structure.

However, they have largely been replaced by other database models, such as relational databases and NoSQL databases, which offer more flexibility and better support for complex relationships.

Common Use-Cases:

- **Organizational Structures:** Managing data in organizational charts where each entity (e.g., employee) has a clear hierarchical relationship.
- **File Systems:** The directory structure of file systems is a classic example of hierarchical data, where folders have subfolders and files.

Examples: IBM IMS, Windows Registry

14. Vector Databases

Vector databases are specialized databases designed for storing and searching vectors which are arrays of numbers representing data in high-dimensional spaces.

They are optimized for similarity search and nearest neighbor queries enabling fast retrieval of similar items based on their vector representations.

These databases are particularly relevant in the field of machine learning and artificial intelligence (AI), where vector representations are commonly used to encode the features of various types of data, including text, images, and audio.

Common Use-Cases:

- **Image and Video Search:** Vector databases enable content-based image and video retrieval by storing visual features as high-dimensional vectors.
- **Recommendation Systems:** By representing users and items (e.g., products, movies) as vectors, vector databases can quickly identify and recommend items similar to a user's interests.

- **Anomaly detection:** By comparing the similarity of new data points to known normal samples, anomalies can be detected based on their dissimilarity.

Examples: Faiss, Milvus, Pinecone.

15. Embedded Databases

Embedded databases are specialized databases designed to be tightly integrated into software applications. Unlike traditional client-server databases that run as separate processes, embedded databases are linked and run as part of the application itself.

By being tightly integrated into the application process, they offer fast data access, small footprint, and simplified deployment.

Embedded databases are particularly useful in resource-constrained environments where a full-fledged client-server database is not necessary or practical.

Common Use-Cases:

- **Gaming:** Saving game states, player progress, and configuration settings directly within the game application.
- **Desktop Applications:** Storing configuration settings, user preferences, and application data locally on a user's machine.

Examples: SQLite, RocksDB, Berkeley DB.

When it comes to choosing a database, there's no one-size-fits-all solution.

The choice of a database depends on your specific use case, data model, scalability needs, and budget.

Thank you so much for reading.

If you found it valuable, hit a like ❤️ and consider subscribing for more such content every week.

If you have any questions or suggestions, leave a comment.

This post is public so feel free to share it.

Subscribe for free to receive new articles every week.

Subscribe

Checkout my [Youtube channel](#) for more in-depth content.

Follow me on [LinkedIn](#) and [X](#) to stay updated.

Checkout my [GitHub repositories](#) for free interview preparation resources.

I hope you have a lovely day!

See you soon,

Ashish



55 Likes · 3 Restacks

[← Previous](#)

[Next →](#)

Discussion about this post

Comments

Restacks



Write a comment...

© 2024 Ashish Pratap Singh · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture